

LEAN_SCOPE_PLAIN

What to Build First (and What to Skip) — In Plain English

Plain-language companion to [docs/planning/LEAN_SCOPE.md](#) . For the exact spec, read that / its PDF.

In one sentence

This is the ruthless cut: in 2 weeks, one developer building with Claude Pro should ship the *honest core* of Helios — governed data, the mix-vs-rate decomposition, one AI tool server, one grounded brief, and one honest benchmark — and deliberately skip the 7-agent fleet, the scheduler, and the grand claims.

Why this matters to you

The full design is a 6–10 week project. You have 2 weeks. This doc tells you exactly where to draw the line so you ship something **correct, defensible, and genuinely impressive** instead of a half-built version of everything. The key insight: a critical "red-team" review found the *value* was never in the apparatus (the agents, servers, scheduler). The value is in four things — keep those, cut the rest.

The big ideas, simply

- **The value lives in four things.** (1) A governed dbt/BigQuery model of the real GA4 funnel. (2) The **mix-vs-rate decomposition** (the technique that makes you look senior). (3) One MCP server + one grounded LLM step (enough to honestly say "AI engineering"). (4) An honest evaluation that beats a naive baseline — and admits what it doesn't prove.
- **Rename the project first.** Not "Autonomous Growth Diagnosis Engine that diagnoses *why*." Instead: **"Helios — a governed, mix-vs-rate funnel-diagnosis engine for GA4."** It

tells you *where* a metric moved and *how much it's worth* — not *why*. This single rename neutralizes the three biggest criticisms.

- **Claude Pro is your pair, not an API fleet.** Pro gives you Claude Code (your dev accelerator) plus modest usage — not the budget to run 7 agents doing many BigQuery round-trips per run. So the design must make **one** LLM call per diagnosis (maybe a second for a self-check). The constraint *forces* the lean design.
- **The code already exists in the docs.** Your 2 weeks is wiring + testing + the brief + the eval — not design.

What you actually do (in order)

The plan splits into two shippable products.

Week 1 — Helios MVP (no LLM yet). Pure, correct, tested analytics + the decomposition.

- Build the trimmed spine: **M0–M2** (repo, BigQuery, dbt, macros, staging).
- **M3** keystone: sessionization + monotonic `reached_*` flags — **golden tests first**.
- **M4**: the core fact tables + a few dimensions. Test: monotonicity, revenue reconciles to the cent, keys unique.
- **M5** trimmed: a **~12-metric** semantic registry (not the full 47).
- `decompose_change` in ~15 lines of plain Python + a significance test, with a golden unit test.
- `diagnose.py` : find the biggest week-over-week move, decompose it by device and channel, attach a dollar figure, print a **templated** brief (no LLM).
- *Done when*: `python -m helios.diagnose` prints a correct, governed mix-vs-rate diagnosis with a real dollar figure.

Week 2 — Helios v1 (this is what you ship). Add the AI and the proof.

- **One MCP server** — `semantic-mcp` or `stats-mcp` — connected to Claude Code. Now "I built an MCP server the LLM can't bypass" is literally true.
- **One grounded LLM brief** — Claude takes the *deterministic* numbers and writes the exec Decision Brief + one recommended next test. One call, fully grounded (numbers are tool outputs, never model-invented).
- **Honest eval** — `injector.py` perturbs a frozen copy of the data, **6–10 scenarios**, a `scorer.py`, and the **naive "largest-absolute-delta" baseline**. Report the number *and* state plainly what it proves and doesn't.
- (*Stretch*) one self-critique pass.
- *Done when*: `python -m helios.run` produces a Claude-written brief from governed SQL, and `python -m helios.eval` beats the baseline.

After 2 weeks — v2 (deferred, not flawed). The remaining servers, a small multi-agent split, the full benchmark + CI gate, the richer semantic layer, experiment design, cohorts/retention (with caveats), memory, a scheduler.

Easy things to get wrong

- **Rebuilding the apparatus anyway.** Resist adding agents, servers, or a scheduler "because the spec has them." On a frozen 3-month dataset they're theater. Cut them.
- **Forgetting the rename.** If you keep calling it "autonomous" and "diagnoses why," you re-import the exact claims the review killed. Say *where* and *how much*, not *why*.
- **Padding the benchmark to 50 scenarios.** The eval's value is honesty + a baseline, not count. 6–10 honest scenarios beat 50 whose labels come from your own algebra.
- **Overclaiming in interviews.** The maturity move *wins* rounds: "My eval proves controlled-attribution accuracy vs a naive baseline. It does **not** prove real-world causal accuracy — the cause lives outside the data. I scoped to what's honest." Saying that is a stronger signal than any 85% number.
- **Building breadth-first.** Go depth-first down the critical path. Finance facts, cohorts, the UI can all wait.

Glossary — the exact words, demystified

- **MVP:** the Week-1 deliverable — correct analytics + the decomposition, no LLM.
- **v1:** the Week-2 deliverable you actually ship — MVP + one MCP server + one grounded brief + honest eval.
- **mix-vs-rate decomposition (`decompose_change`):** the technique that splits a metric's movement into "the traffic mix shifted" vs "behavior actually changed" — how you avoid the Simpson's-paradox mistake real analysts make.
- **Simpson's paradox:** when a blended number moves only because the *mix* of segments changed, not because any segment got better or worse. A naive "biggest segment delta" gets the wrong root cause here.
- **MCP server:** a tool service the LLM calls and physically cannot bypass — so it can't hand-write SQL (→ 0 hallucinated columns).
- **grounded LLM brief:** Claude writes prose, but every number is a tool output it didn't compute.
- **governed SQL:** SQL the model never authors by hand — it composes named metrics through the semantic layer.
- **semantic layer (~12 metrics here):** the registry of approved metric definitions. The full design has 47; the lean cut keeps ~12.

- **naive baseline:** the dumb "largest absolute segment delta" method your eval must beat.
- **M0–M5 / M6 / M7 / M10:** milestone IDs from the full build map; the lean plan builds the M0–M5 spine plus thin slices of M6/M7/M10.

What's IN vs OUT (the one-glance table)

IN (the spine)	OUT (cut or deferred)
dbt marts: sessionization → funnel → <code>fct_funnel</code> / <code>fct_daily_funnel</code> / <code>fct_orders</code> + ~4 dims	The 7-agent FSM (→ one grounded LLM call)
~12 governed metrics	~35 of the 47 metrics; CAC proxy, channel-revenue
<code>decompose_change</code> + significance, golden-tested	4 of 5 MCP servers (warehouse, experiment, report)
One MCP server (<code>semantic</code> or <code>stats</code>)	Autonomy / scheduler / "always-on"
One grounded LLM Decision Brief	Memory / vector store / suppression
Honest eval: 6–10 scenarios vs naive baseline	Cohorts / retention / RFM / forecasting
This scoping decision (an interview asset)	The 50-scenario benchmark + CI gate
	The words "autonomous" and "diagnose why"

When to open the real doc

Open `pdf/docs/planning/LEAN_SCOPE.pdf` (or the `.md`) when you need the exact day-by-day 2-week table, the precise "keep vs cut" reasoning per item, or the interview-ammunition talking points. This companion gives you the line in the sand; the real doc gives you the full justification for where it sits.